Betrina Makaika efp-bsc-com-ne-26-25

Assignment 1 — Theory Paper

IoT Telemetry Pipeline & Dashboard

A. Serialization Choices

Different serialization formats are used in the system because each part has different needs. YAML is used for sensor configuration because it is simple, human-readable, and easy to edit. YAML focuses on key-value structures and uses spacing and indentation instead of many brackets or symbols, making configuration files clean and easy to understand. It also supports comments, which helps technicians explain settings inside the file. YAML is commonly used in configuration management because it can store complex data in a format that is easy for both humans and machines to read.

Protocol Buffers (Protobuf) are used between sensors and the server because they are fast and efficient. Protobuf uses a compact binary format that reduces message size, saves bandwidth, and improves performance. This is important because many sensors may send readings at the same time. Protobuf also uses a schema defined in a .proto file, which ensures that both the sensor and server understand the same message structure. While YAML is best for human editing, Protobuf is best for machine communication.

JSON and XML are used for the public REST API because they are standard formats supported by many systems. A mobile app can easily use JSON, while desktop applications may prefer XML. REST APIs normally use HTTP methods such as GET, POST AND DELETE and return data in JSON or XML. These formats are human-readable, easy to debug, and allow different systems written in different programming languages to communicate easily.

B.**Web Services Protocol**

REST is suitable for the historical telemetry API because it is resource-based and works well with HTTP. In this system, sensors and readings are treated as resources such as /sensors and /sensors/{id}/readings. HTTP methods match the required actions: GET retrieves data, POST adds data, and DELETE removes data.

REST is also stateless, meaning every request contains all the information needed for processing. This makes the system simpler and easier to scale because the server does not need to remember previous requests.

SOAP would not be a good choice because it is more complex and mainly depends on XML messages with extra overhead. RPC focuses more on calling functions remotely instead of managing resources. REST is simpler, lighter, and easier to integrate with web and mobile applications, making it a better choice for this telemetry system

C. **AsyncIO vs Threading**

This system is mainly I/O-bound because most operations involve waiting for network communication, database access, or file operations. The server spends more time waiting for sensor data and sending responses than doing heavy calculations. Because of this, AsyncIO is more suitable than using threads.

Coroutines in AsyncIO are lightweight and use much less memory than threads. Thousands of sensor connections can be handled efficiently without creating thousands of threads. When a coroutine reaches an I/O operation, it uses await to pause and give control back to the event loop. The event loop can then continue running other tasks while waiting for the operation to finish. This improves efficiency and responsiveness.

AsyncIO also reduces context-switching overhead compared to threads. Thread switching is handled by the operating system and is slower, while coroutine switching happens inside the program and is faster. Another advantage is that AsyncIO avoids many synchronization problems such as race conditions and deadlocks because tasks cooperate within a single event loop.

Threads or multiprocessing would be better for CPU-intensive tasks such as machine learning, image processing, or complex calculations. Since this telemetry system mainly handles network communication, AsyncIO is the better choice.

D. HTTP Mechanics

Content negotiation is important in the REST API because different clients may need different response formats. Clients use the Accept header to request formats such as JSON, XML, or YAML. The server checks this header and returns data in the requested format. For example, a mobile app may request JSON while a legacy desktop tool requests XML. This allows one API to support many different clients.

Cookies are also important in this system. HTTP is stateless, meaning the server does not automatically remember previous requests from the same client. Cookies solve this problem by storing small pieces of data in the client's browser or application and sending them back with future requests.

In this telemetry system, cookies can help create a personalized farmer experience. The server can remember a farmer's preferred sensors, dashboard settings, or favorite data format. This improves usability because users do not need to configure the same settings every time they connect to the system.

References

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine.

Python Software Foundation. “asyncio — Asynchronous I/O.” Python AsyncIO Documentation

YAML Organization. “YAML Ain’t Markup Language (YAML™).” YAML Official Documentation

Mozilla Developer Network (MDN). “HTTP Cookies.” MDN HTTP Cookies

Mozilla Developer Network (MDN). “Content Negotiation.”MDN Content Negotiation

Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O’Reilly Media.

Mozilla Developer Network (MDN). “WebSocket API.” MDN WebSocket API